e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:05/May-2026 Impact Factor- 8.187 www.irjmets.com

A LITERATURE SURVEY ON PRIVACY-PRESERVING MULTI-AGENT RAG SYSTEMS WITH AN INTELLIGENT TAG-INFERENCE ROUTING SYSTEM

Jaidatt Kale*1, Harshita Jain*2, Atharva Korwar*3, Kanchan Jojare*4

*1,2,3Department Of Computer Engineering, DPCOE, Pune, Maharashtra, India.

*4Professor, Department Of Computer Engineering, DPCOE, Pune, Maharashtra, India.

DOI: https://www.doi.org/10.56726/IRJMETS98584

ABSTRACT

As the number of question-answering systems based on Large Language Models (LLMs) has increased rapidly, a problem that remains poorly understood and largely overlooked is the fact that numerous current platforms routinely pass confidential documents, sensitive database records, and other user data to external cloud-based APIs with no control over data sovereignty at the granular level of the question. The survey provides a comprehensive overview of the underlying research studies and the latest developments in this area that form the basis for the suggested privacy-oriented multi-agent web app. The system includes an intelligent dual-layer LLM routing mechanism that can classify queries according to user-defined classification rules. While the queries that are tagged as sensitive by the platform's automated detection feature are sent to a locally deployed LLM (Ollama), the non-sensitive queries are sent to a configured cloud-hosted LLM service. This design ensures that personal data stay on the device of the user and are not moved to the outside.

The review explores five key technical areas: (1) Retrieval-Augmented Generation (RAG) systems and their evolution from a single-agent system to multi-agent systems; (2) distributed and privacy-preserving methodologies in RAG; (3) Natural Language-to-SQL (NL-to-SQL) conversion methods, which utilize compact locally-executable LLMs; (4) orchestration in multi-agent systems, as well as design patterns for agentic architectures; and (5) compound AI architectures for enterprise deployment at scale. The survey highlights significant gaps in existing solutions, especially query routing solutions to ensure privacy, by integrating and comparing these research areas. It thus sets out the proposed scheme as a specific remedy for these shortcomings. The study also offers a solid academic justification of the architectural decisions for the system and highlights some research challenges for privacy focused agentic AI.

Keywords—Retrieval-Augmented Generation; Multi-Agent Systems; Privacy-Preserving AI; Natural Language to SQL; Intelligent Routing System; Tag Inference; Local LLM; Query Orchestration; Agentic AI.

I. INTRODUCTION

With the growing adoption of Large Language Model (LLM) Question Answering (QA) systems across different industries, academic institutions, and personal computers, the scope for data breaches has significantly expanded. Cloud-based processing models are commonly used in most cloud platforms, open-source or not, where all the contextual information retrieved is sent to external API services for computation based on user prompt. This model can be used for requests for publicly available information that is not sensitive, but may not apply when the information requested is information to which a person's name is attached (personally identifiable information, PII) or confidential financial information, proprietary database structures or restricted organisational documents.

While research has been carried out on privacy-preserving machine learning, federated learning, and differential privacy, the problem of achieving privacy with the routing in LLM-based QA pipelines has not been studied as extensively. Traditional Retrieval-Augmented Generation (RAG) systems [1, 3] typically address query routing as a way to boost system performance and efficiency while enforcing privacy constraints are not always considered. At present, there is no widely accepted alternative that allows users to explicitly specify, when a query is executed, whether it is processed locally or on some remote cloud based infrastructure.The proposed approach for mitigating this is based on providing a smart routing mechanism which reads the query tags in the user requests. These structured identifiers—like @filename.f for document based resources, and @dbname.d for database related resources—allow to infer the associated data source and identify and choose

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:05/May-2026 Impact Factor- 8.187 www.irjmets.com

the correct layer used to run the LLM as a result of the privacy characteristics. If a tag is recognized, the routing component determines that the request is deemed to contain sensitive data, and it will only route it to a locally deployed LLM using Ollama so that sensitive data will not travel to and from other networks and servers. If there are no inferred tags specified then requests go to a cloud-based API that is specified in a parameter. The dual-layer routing model is based on a multi-agent model comprising five specialised agents—the Router Agent, Document Agent, Database Agent, General Agent and Orchestrator. The data receives coordination by a FastAPI backend, and ChromaDB works as the vector storage to achieve data operations and PostgreSQL as the relational database layer.

This literature survey will address the structured overview of research foundations for each and every major architectural component of the proposed system. The review is broken up into a number of sections. In Section II, the authors discuss the evolution of Retrieval-Augmented Generation (RAG) architectures, from monolithic, single-agent, systems to specialized, multi-agent systems. In Section III, the latest developments in Privacy Preserving and Distributed RAG methods are examined. Section IV addresses the Natural Language-to-SQL (NL-to-SQL) translation techniques, which are an important functional ability provided by the system's Database Agent. Section V explores possible ways of multi agent orchestration as well as the general idea of agentic system design. Using artificial intelligence in businesses is possible, and the focus of Section VI is compound AI architectures for enterprise scale implementation. Section VII brings together the key gaps found throughout the reviewed literature which were used as the basis for designing the proposed framework. Last but not least, conclusions from the survey are provided in the last section (VIII).

II. RETRIEVAL-AUGMENTED GENERATION: FROM SINGLE-AGENT TO MULTI-AGENT ARCHITECTURES

A. Foundational RAG Systems

Lewis et al. [1] first formalised the idea of improving LLM-generated responses with dynamically retrieved external knowledge, which was later dubbed as Retrieval-Augmented Generation (RAG). This initial article showed that by pairing a dense passage retrieval system with a generative language model, this kind of system is able to benefit significantly from the use of knowledge for open domain question answering, achieving significantly better result than purely parametric models or specialised task specific systems. The design's idea is to use a neural retriever to find the relevant passages from an indexed corpus (e.g. a Wikipedia dataset stored in a dense vector index) and then add the retrieved content at the end of the user query and pass it on to the generative model. This latter architectural distinction for retrieval and generation laid down the building blocks used by later RAG systems.

Although for an important feature, the initial architecture is limited by its reliance on a single retrieval agent. According to Salve et al. [2] however, single-agent RAG systems must store a vast and varied schema awareness, as well as domain-specific reasoning capabilities for each data source they support, in addition to a comprehensive few-shot prompting example set. This has to be data-specific, introduces a significant token overhead and decreases the effectiveness if the data is in a heterogeneous data store, e.g. a relational database, a document repository or a semi-structured information source. As such, the one agent can be a scalability and performance blocker as it is not able to adapt retrieval and query generation pattern to specific structural features of each data source category.

B. Multi-Agent RAG Systems

Due to the limitations of monolithic designs, there has been a growing trend towards multi-agent RAG designs, where each agent is tasked with processing a different type of data source. The collaboration multisemantic RAG architecture proposed by Salve et al. [2] is one of the closest works and presents a multi-agent architecture with specialized agents such as MySQL Agent, MongoDB Agent, Neo4j Agent and ElasticSearch Agent. A specific few-shot prompting strategy is used for each query to a database system to ensure that a query is generated according to the domain of the database system that it is being sent to. Instead of just performing the query themselves, these agents send the conceptually generated queries to a central execution layer; a different agent then combines all of the outputs of all agents, into one output for the end user.

Owing to this decentralised approach to responsibility there are a number of significant benefits. Since each agent will need only prompts, schema context and examples for a single environments of a database, token

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:05/May-2026 Impact Factor- 8.187 www.irjmets.com

consumption is reduced. There is also an improvement in query accuracy, as agents learn how to reason for optimal support in the structure and operation of the system(s) they have been assigned to. In addition, the architecture is extensible, meaning that adding new or new-to-the-farmers market data platforms can be accomplished without incurring any change in other aspects. This is an expression of the modularity principle that is formalised as a specialisation relationship by Salve et al.—(S_{agent} = f_{specialisation}(D_{data_type}, A_{agent_type}))—which does capture this modularity approach and offers theory for the multi-agent decomposition used in the proposed framework.

This is the direct application of this architectural model to the proposed system. A dedicated Document Agent can be used to process the retrieval using the fragments of the documents embedded in ChromaDB, including through PDF, DOCX, CSV and Processing files. A Database Agent handles the NA-to-SQL translation of data sources (PostgreSQL) and a General Agent handles the open-domain requests of a cloud-hosted LLM. The key improvement that is added to the architecture outlined in Salve et al's description is the addition of privacy-aware routing logic. In this architecture, agents are not just based on the type of the data source but also based on the information that can be obtained from the data source that can be safely processed by an external LLM over the cloud or processed entirely locally.

C. Advanced RAG Variants

Besides the shift towards multi-agent architectures, the RAG research area has brought several innovations in the framework of architecture directly relevant to the proposed architecture. Self-RAG [3] introduces a new reflection token-based self-reflective retrieval approach in which the retrieval mechanism is also trained to determine when to retrieve and what information to retrieve, as well as how to evaluate generated answers. This adaptive retrieval paradigm is especially relevant to the proposed system, as it is this that will guide both the design of the agent-tracing aspect of the system and the design of the reasoning-transparency aspect of the proposed system. The system makes retrieval actions, the LLM processing these actions, and related privacy metadata, explicit in a collapsible AgentTrace interface, rather than hiding them from the user.

Speculative RAG [4] is a dual-model approach with a lightweight model – the specialist model – and larger model – the general purpose model. By this model, the more compact language model produces numerous responses based on the intersection of the retrieved documents and the larger language model validates and outputs the best one in a single stage of evaluation. Experimental results prove an advantage of up to 12.97% in accuracy and latency improvement of about 51% in performances carried out on benchmarks (including TriviaQA, MuSiQue, PubHealth). The idea of having two types of LLM, a fast one and a slower, yet more powerful one, corresponds neatly with the two different LLMs that every system is recommended to use. The specially trained LLM is deployed as the LLM specialist on the premises, while the generalist LLM is hosted in the cloud, to handle more general open domain interactions.

Hierarchical RAG frameworks, such as HiRAG [5] and ArchRAG [6] offer multiple levels of retrieval, refining contextual information gradually along increasingly cohesive document structures or interconnected groups of knowledge. The current implementation of the proposed system uses a flat top-(k) retrieval strategy whereit retrieves document chunks from a ChromaDB for each top document retrieved, but these hierarchical retrieval approaches are a potential avenue for future improvement for better precision and context for retrieval over large-scale document sets.

III. PRIVACY-PRESERVING AND DISTRIBUTED RAG SYSTEMS

A. Privacy Vulnerabilities in RAG Pipelines

Private data sources have been integrated into Retrieval-Augmented Generation (RAG) systems, giving rise to new security and privacy concerns that are now subject of academic research. Zeng et al. [7] offers a detailed account of the vulnerabilities of RAG systems concerning privacy issues, categorizing them into two types. It starts with membership inference attacks, where attackers try to know if a document is in the knowledge base of the system. The second is data extraction attacks, in which malicious crafted queries are used to steal sensitive information from the model or retrieval pipeline. Xu et al. [8] also suggest that the common RAG architecture (which usually includes a dense retriever, a vector database, and a generative LLM) faces a distributed trust challenge as each component is hosted by different parties and sensitive information is processed through all components with only weakened access-control guarantees.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:05/May-2026 Impact Factor- 8.187 www.irjmets.com

These security concerns mirror closely the threat model targeted by the proposed model. Any processing done on the cloud may expose the likelihood of membership inference and data extraction attacks when users upload sensitive documents or set up associations with proprietary PostgreSQL databases. This is mitigation by way of the architectural design of the proposed system. The intelligent routing mechanism ensures that information originating from a tagged source can never be sent to an external cloud-based LLM API by inferring a structured tag within the user query. Embedded documents of sensitive data sources are stored locally in the Ollama-story specific instance of ChromaDB, and NL-to-SQL operations are performed locally in the Ollama-story specific NL-to-SQL inference framework.

B. Distributed and Privacy-Preserving RAG Frameworks

Xu et al. [8] propose a distributed Retrieval-Augmented Generation (RAG) system, which splits retrieval to multiple independent nodes to reduce each individual node's access to the full private data set. The Distributed RAG is their approach that partitions document embeddings with privacy considerations, preventing one retriever from knowing the entirety of the knowledge store. To enhance privacy protections, the framework introduces differential privacy methods for retrieval outputs, minimizing the likelihood of adversarial inference from the retrieved passages themselves.

This is particularly relevant for the proposed framework as it formally endorse support for the rationale behind localised processing — the provision of processing information within an environment separated from any external communication from networks. The proposed system adopts this idea by introducing a binary routing strategy that signals whether the processing is performed at the local or at the cloud-bound LLM service side. Instead of proposing an architecture like Xu et al. that was distributed and cryptographically enhanced, the proposed architecture highlighted here focuses on simplicity, transparency, and ease of auditing. This allows for simple implementation and the explainability for end users without compromising to a great extent on privacy guarantees suitable for single user, desktop device or server deployment.

The design space is further illustrated with additional works on federated RAG systems [9] and privacy-preserving retrieval approaches [10]. The main federated approaches for collaborative training of retrieval models involve either teacher–student knowledge distillation or using homomorphic encryption on model gradients to protect sensitive data while simultaneously preserving the model's quality. While such methods are very promising in a multi-organisation setup, where multiple independent organisations wish to improve their common retrieval framework, they also make for significant computational and communication burdens. This overhead is not necessary when the proposed system is to be deployed for a single user. For this reason the local-routing strategy pursued in the architecture appears to be a more feasible and effective option for the context under consideration; federated approaches are included as design possibilities with benefits for future extensions to the deployment, particularly multi-user and multi-tenant applications, but ultimately are not as central to the solution as choices for local-routing.

C. Local LLM Deployment for Privacy

An ongoing line of research has looked at using locally-hosted open-source LLMs as an effective method to safeguard data privacy. In addition, Naveed et al. [11] provide a comprehensive overview of LLM architectures, segmenting them between models that can be deployed locally like Mistral, Llama, Zephyr, and Phi-2, and those hosted externally like GPT-4 and Gemini API LLM. Locally deployed models are particularly useful in sectors like healthcare and finance, where privacy regulations and compliance standards mandate that sensitive data stay within organizational limits. Salve et al. [2] underscore the importance of such models in such sensitive fields where privacy regulations and compliance standards require the containment of sensitive information within organizational boundaries. This allows data to be primarily processed on-premises, preventing the transmission of confidential data to third-party services. Even so, local deployment has its significant drawbacks and issues such as higher computational resources and lower performance on complex reasoning tasks compared to large models hosted on cloud services.

Qu et al. [12] also explore the application of compact LLM packages for local deployment in reference with the Electronic Health Record (EHR) systems which are more closely requested by privacy constraints. Their results show that, with a proper fine-tuning, 7B parameter models can successfully meet the clinically relevant criteria

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:05/May-2026 Impact Factor- 8.187 www.irjmets.com

on structured data question-answering problems, preserving on-premises data sovereignty. These results directly support the architectural decisions made for the proposed system. More specifically, it sends privacy-related and structured information queries to a local Ollama instance with the local LLM handling these tasks, which includes generating SQL and synthesizing documents chunks. More difficult open domain queries, on the other hand, are served by cloud-based LLM services, which have superior general reasoning capacities.

IV. NATURAL LANGUAGE TO SQL TRANSLATION

A. Evolution of NL-to-SQL Methods

Natural Language-to-SQL (NL-to-SQL) is also known as Text-to-SQL, is the process of generating an SQL query given a natural language request. This is a core element of the proposed system's Database Agent, an entity designed to translate prompts, like “How many orders were placed last month in @shopdb.d?”, into "safe and executable" PostgreSQL queries. NL-to-SQL has gone through several phases of methodological advancement. Early solutions relied largely on manually written grammar rules and template matching method of parsing [13] which required intensive manual efforts and were not easy to generalize for unknown database schemas. Subsequent improvement was inspired by transformer-based sequence-to-sequence models like RAT-SQL [14] and LGESQL, which added relation-aware self-attention mechanisms and graph neural networks to capture the structural correlations between natural language expressions and the elements of a database schema.

B. LLM-Based NL-to-SQL and Privacy Implications

Since the large language models were introduced, the state of NL-to-SQL systems has taken a significant leap forward. Prompt-driven approaches like DIN-SQL [15] and DAIL-SQL have shown that models such as GPT-4 can attain higher than 84% execution accuracy on Spider when provided with database schema information and properly designed few-shot examples. Although this speedup improves performances, Pourreza and Rafiei [13] note that relying on 3rd-party, purpose-built cloud-hosted LLMs for NL-to-SQL operations automatically creates significant privacy risks. There are regulatory and confidentiality-related concerns that prevent many organizations, especially large businesses, from passing details from their customer databases or company data to third parties providing a model. This is directly relevant to the architectural choice for all queries on a database be handled by a locally-deployed LLM instead of an external cloud service.

To overcome small-sized open-source models, Pourreza and Rafiei [13] present DTS-SQL: a decomposed fine-tuning framework designed to minimize the performance gap between smaller-sized (7B-parameter) open-source models and large-sized proprietary ones. They do so by creating a 2 step approach: one to link all the relevant tables and columns to the user query, and a second to generate SQL based on the selected schema context. With 7B’s DeepSeek model, DTS-SQL achieves an execution accuracy of 60.31% for the BIRD hold-out benchmark, the best publicly reported performance by 7B-scale methods at the time of writing.

The proposed system is designed around its Database Agent, and the decomposition approach described in DTS-SQL is very salient to the design of this component. The architecture features a separate stage for schema-context generation, where the linked PostgreSQL database is automatically queried using the information_schema interface, and the information collected (schema metadata) is represented as a JSON-based entity. This structured schema context is then integrated into each NL-to-SQL prompt passed to the locally hosted Ollama model, and allows the LLM to create more accurate and contextually relevant SQL queries.

C. Schema Context and SQL Safety

The ability of schema representation in the LLM prompt is significant in supporting SQL generation accuracy in the vision of the NL-to-SQL research area. Recently, Retrieval-Augmented Text-to-SQL approaches [16] have proposed to intelligently retrieve only schema parts that are essential to the user query, as a way to decrease prompt length and enhance alignment of the generated SQL statements with the queried schema. In the framework to be developed, this is a part of the schema-context generation pipeline being implemented. If a new PostgreSQL database is attached, it automatically discovers table names, column names, datatypes, nullable attributes, enumeration values, and relational constraints from the database's metadata in the information_schema. This raw schema information is then translated into a structured JSON context object that includes summaries and descriptions of the tables, descriptions of the columns, guidance for how to handle

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:05/May-2026 Impact Factor- 8.187 www.irjmets.com

column types, and rules for generating queries using the schema. This creates a schema context that is stored locally and attached to all subsequent NL-to-SQL prompts that are made to the database, such that the local Ollama model can work in a detailed schema-aware way.

SQL safety validation is another feature that is growing in importance in the NL-to-SQL literature. Ziletti and D'Ambrosi [16] highlight that one important responsibility for an NL-to-SQL system is ensuring there are mechanisms to prevent generation of destructive queries, such as those containing DROP, DELETE, TRUNCATE, ALTER, CREATE, INSERT, and UPDATE operations that can modify or drop data if the model fails to interpret user intent or if the input data is unclear. The recommended way to address this risk is to allow a list of SQL keywords which are considered "safe" to use, by setting the environment variable RESTRICTED_SQL_KEYWORDS. An exception that has restricted operations in it is intercepted before it is executed, and a clear exception is thrown instead. This is a defence-in-depth approach, as the local LLM is specifically programmed to only create SELECT statements, and sql_validator.py is an extra layer of validation to prevent potential risks when running this on operations.

V. MULTI-AGENT ORCHESTRATION AND AGENTIC SYSTEM DESIGN

A. Cognitive Architectures for Language Agents

Its architecture is based on recent research on cognitive architectures for language-based agents. Sumers et al. [17] propose a comprehensive theoretical model for language agents that classifies them based on dimensions of memory (in-context memory, external memory, and parametric memory), action (retrieval, code execution, and communicating with other agents), and decision-making (planning, reasoning, and reflecting). They show that given a variety of task contexts, monolithic systems face essential constraints of context-window capacity and domain specialisation when a single system tackles multiple goals. These restrictions are directly relevant to the proposed system's decomposition into five specialised agents, covering each of its operational roles.

Talebirad and Nadiri [18] also explore how to coordinate LLM agents with lower level agents for multi-agent LLM systems, and categorize the coordination models to be hierarchical coordination, peer-to-peer coordination, and hybrid approaches that mix elements of both models. The proposed framework is of a hierarchical orchestration approach. In this arrangement, the Orchestrator Agent handles the entire query-processing process by first contacting the Router Agent to access private and sensitive information and then passing the query on to the appropriate specialised component of the system: the Document Agent, Database Agent, or General Agent. After processing, responses are subsequently returned via the Orchestrator, which enriches the response with other metadata such as the LLM used for generation, privacy classification, response latencies, and traces of agent execution.

B. The HAWK Multi-Agent Workflow Framework

In a wider architectural context, the HAWK use of cases [19] is a valuable reference model for orchestrating structured multi-agent workflows. HAWK is structured in five layers: User, Workflow, Operator, Agent and Resource, with sixteen standardised interfaces to foster modularity, extensibility and interoperability between various system components. The framework also contains an Adaptive Scheduling module that can perform intelligent task allocation aware of context, and a Unified Resource Abstraction layer provides consistent abstraction of interaction with various models, services, and data sources.

The suggested architecture of the proposed system shows that this is a layered system with remarkable conceptual congruence. The github frontend is based on the React component and HAWK's User layer will take care of the functionality of the FastAPI gateway, which will perform the role of the Workflow layer. The Agent layer is implemented by the specialised agents: router_agent.py, document_agent.py, database_agent.py, general_agent.py, and orchestrator.py. Similarly, the LLM abstraction modules (llm_local.py and llm_cloud.py) together with the data registries align with the Resource layer. Consistent data contracts are present throughout all system boundaries using interface standardisation as Pydantic schema definitions like ChatRequest, ChatResponse, AgentTrace, DocumentRecord, and DBRecord.

One of the major differences between HAWK and proposed architecture is how privacy is handled. In HAWK, the assignment of the agents depends mainly on the characteristics of the tasks and availability of the

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:05/May-2026 Impact Factor- 8.187 www.irjmets.com

resources. In contrast, the proposed system builds privacy-awareness into its architectural foundations, with the agents used and the decisions made by LLMs made explicitly based on the inferred privacy sensitivity of the data a query accesses using the system.

C. Multi-Agent RAG for Technical Support and Communication Systems

In technical knowledge-based systems, multi-agent Retrieval-Augmented Generation (RAG) systems have been explored, with particular emphasis on relevance-feedback mechanisms and collaborative communication methods like Reflection and Planning [20]. Their experiments show that the accuracy rate can be as high as 55% higher with different types of multi-agent interactions than with simpler single-agent ones on technical support question-answering tasks. Results indicate that multi-agent decomposition strategies are empirically supported as effective. Equally important, their work also shows that smaller, open-source models like Llama3 can replicate the results of more powerful proprietary models like GPT-4 with well-crafted prompts. The proposed solution to address privacy concerns for handling queries by deploying an Llama3 model locally within a specific network or region aligns directly with achieving this outcome.

Jiang et al. [21] introduce a multi-agent optimisation system for 6G communications, with three coordinating modules: Multi-Agent Data Retrieval (MDR), Multi-Agent Evaluation and Reflection (MER), and Multi-Agent Collaborative Planning (MCP). While the application space is quite distinct from the proposed framework, the concepts for architecture are quite generalizable. Explicitly dividing responsibilities for retrieval, evaluation and planning into separate functions performed by agents with dedicated communications between agents provides a general template for agent decomposition. In the proposed system, the Router Agent is similar to the role of MDR that is to build up the most appropriate retrieval pathway for a query; the Orchestrator Agent is similar to the role of MCP, which is to coordinate the flow of execution and to create the final unified response.

VI. COMPOUND AI SYSTEMS FOR ENTERPRISE DEPLOYMENT

A. Blueprint Architecture for Enterprise AI

Kandogan et al. [22] propose a reference architecture for compound AI systems in enterprise settings, which emphasizes the inclusion of LLMs and the seamless integration of them and a variety of data sources in enterprise environments, without compromising reliability, efficiency, and usability. They introduce useful architectural abstractions such as the Agent Registry for keeping track of available agents and their capabilities; a Task Planner which breaks down user requests into a set of tasks that can be executed by the agents; a Task Coordinator that monitors the quality of execution and the constraints of the resources; and a Data Operator layer that offers a unified interface for engaging with relational databases, document repositories and graph-based structures.

Many of these ideas are covered in the proposed framework in a more specialized single user deployment scenario. The combined effect of the "document_registry.json" and "database_registry.json" files is to serve as a sort of Agent Registry database, defining what data sources are available, query tags that match those data, contexts for executing queries and operational metadata. The Router Agent acts like the Task Planner, and it examines the embedded tag to decide which specialised agents are to be activated, and the appropriate execution parameters. The Orchestrator plays a role similar to that of the Task Coordinator by marshaling together intermediate results and adding execution-relative metadata to the delivered response. At the same time, functionality similar to the Data Operator abstraction presented by Kandogan et al. is achieved by combining the two pipeline modules, `document_rag` and `database_rag`.

Hence, the architecture offered by Kandogan et al. is theoretically solid for implementation of the decomposition strategy used in the proposed system. Concurrently, agent orchestration to include privacy-sensitive query routing defines a clear extension beyond the original blueprint, embracing privacy sensitivity as a key consideration in the selection of agents and LLMs.

B. LLM Routing and Unsupervised Query Routing

Bringing a different kind of LLM performance and cost to perform the same task is a real challenge that is being actively explored. Shnitzer et al. [23] conceptualise LLM routing as an optimisation problem over a portfolio of models that exhibit varying trade-offs between capability and computational expense. They use a routing model

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:05/May-2026 Impact Factor- 8.187 www.irjmets.com

that is trained on the benchmark datasets to classify the query into the types of queries the various LLMs are designed to model, and then route the query to the appropriate LLM. In a similar vein, Mu et al. [24] propose an unsupervised routing strategy for a Retrieval-Augmented Generation (RAG) system, employing clustering of query embeddings to categorize queries into distinct groups for each retrieval method to maximize the likelihood of finding relevant documents for a specific query without requiring labelled query training data.

Compared to these statistically derived routing techniques, the routing mechanism proposed in the current framework is deliberately more primitive and intuitive, yet provides more interpretability and privacy assurances. The proposed system would use an intelligent tag-inference mechanism to make deterministic routing decisions in the absence of learning routines based on the structured query annotations, which would also require a training set of policies potentially including sensitive queries. In this way, the routing behaviour continues to be consistent, predictable, and completely user-controlled. In concept, the mechanism works as a query processing level the mandatory access control principle used in operating systems security concept and extended to the LLM-based query routing domain.

C. Vector Retrieval and Embedding Stores

ChromaDB will be the vector storage that comes as part of the proposed vector database package which will form part of an growing class of embedded vector databases designed for use in situations that require local or privacy-focused implementation. The design of the persist embeddings as on-disk collections by collection identifiers, which will be supported by the proposed system with document hashes, makes it especially appropriate for single user settings with retrieval from multiple document sources. Furthermore, the ability to create document chunk embeddings without relying on external cloud service connections during the embedding process, as provided by embedding models like nomic-embed-text compatible with ollama, is permitted.

The proposed architecture is thus able to represent an end-to-end local processing pipeline for the document-oriented RAG operations, including ingestion of documents, parsing, chunk segmentation, generation of embeddings, indexing, retrieval, and generation of answers by LLM. This exhaustive local-first approach is a key design feature of the system and supports privacy considerations throughout the retrieval and generation process, often not featured explicitly in current RAG research papers.An integrated approach that addresses privacy from start to finish, as opposed to most papers in the RAG research literature.

VII. IDENTIFIED LITERATURE GAPS AND CONTRIBUTIONS OF THE PROPOSED SYSTEM

The survey of the preceding five technical domains reveals a consistent and significant gap: while the individual components required for privacy-preserving multi-agent QA systems are well-studied in isolation, no prior system integrates them into a cohesive, user-facing application with privacy-aware routing as a first-class architectural concern. Table 1 summarises the key gaps identified.

Table 1: Identified literature gaps and architectural responses of the proposed system.

Research Domain	Gap in Existing Literature	System Response
Multi-Agent RAG	Agent specialisation driven solely by data source type; no privacy sensitivity dimension in routing decisions.	Tag-based routing adds a privacy axis to agent selection; sensitivity (not just data type) determines LLM tier.
Privacy-Preserving RAG	Existing privacy RAG approaches (federated, differential privacy) are designed for multi-party, high-overhead settings; no lightweight local-routing mechanism for	Local LLM tier (Ollama) processes all sensitive queries; zero network transmission; full-stack local pipeline from embedding to generation.

e-ISSN: 2582-5208
International Research Journal of Modernization in Engineering Technology and Science
(Peer-Reviewed, Open Access, Fully Refereed International Journal)
Volume:08/Issue:05/May-2026 Impact Factor- 8.187 www.irjmets.com

Research Domain	Gap in Existing Literature	System Response
	individual users.	
NL-to-SQL	State-of-the-art NL-to-SQL systems depend on proprietary cloud LLMs (GPT-4), making them unsuitable for private database querying.	LLM-generated schema context + local Ollama NL-to-SQL pipeline; SQL safety validator blocks destructive operations.
LLM Query Routing	Existing routing systems optimise for performance/cost trade-offs; privacy is not a routing criterion; routing policies are learned and opaque.	Intelligent routing system that inferences query tags; deterministic, auditable, and fully user-controlled; functions as mandatory access control at query level.
Compound AI Systems	Enterprise AI blueprints do not address user-level privacy enforcement; data governance is a secondary concern rather than a primary architectural driver.	Privacy-by-design: every architectural decision (local ChromaDB, local embeddings, intelligent tag-inference routing, SQL validator) is motivated by privacy enforcement.

In summary, The proposed system's primary scientific contribution is the introduction of privacy-aware query routing as a first-class concern in multi-agent RAG system design. While each component—RAG retrieval, NL-to-SQL translation, multi-agent orchestration, local LLM inference—is individually well-studied, their integration into a unified system where privacy sensitivity governs the entire execution path represents a novel architectural contribution not addressed by any prior work surveyed.

VIII. CONCLUSION

This survey offers a thorough examination of the research underpinnings that drive the suggested framework: an intelligent query routing system based on a privacy-preserving multi-agent Retrieval-Augmented Generation (RAG). The survey has highlighted five major technical challenges across the landscape of question-answering systems based on LLMs, with privacy-aware query routing as a common and significant missing component of the systems' design.The survey has identified five main technical areas, including RAG architectures, privacy-preserving retrieval techniques, Natural Language-to-SQL (NL-to-SQL) translation, multi-agent orchestration strategies, and compound AI system design, and privacy-aware query routing has been identified as a critical and significant gap in current LLM-based Q&A systems.

The original RAG formulation by Lewis et al. [1] laid the groundwork for the current paradigm of retrieval enhanced language systems. This paradigm was further extended by subsequent multi-agent developments [2] that leverage on the use of specialised agents designed to fit in the context of heterogeneous data environments. Researchers have formally defined the risks of disclosure of sensitive information in RAG [7, 8], and compact deployment of locally hosted LLMs have been shown to achieve competitive NL-to-SQL performance in DTS-SQL [13] research. Concurrently, compound AI architectural frameworks [22] provided scalable orchestration patterns for enterprise level intelligent system. The proposed framework combines all these research directions in a common framework where the inferred privacy sensitivity of a query dictates the entire execution workflow from LLM selection, retrieval approach, to response generation.

There are a number of questions that could be explored in future studies. First, it is possible to extend the existing binary routing strategy for local and cloud processing to a more fine-grained privacy-tier model that could allow for different locally deployed models to be matched to different sensitivity requirements and computational constrains. Secondly, the existing routing mechanism requires users to provide structured tags in their query, which might need a lot of user effort, but if lightweight local classifiers could classify

automatically and add the classification to the query, then the user effort can be reduced while maintaining privacy guarantees. Third, enabling multi-user or multi-tenant deployments would likely benefit from similar mechanisms to federated RAG approaches [9] to enable collaborative retrieval improvements without centralising sensitive information. All of these problems present an interesting research field that lay at the intersection of privacy engineering, agentic AI and information retrieval systems.

IX. REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W. Yih, T. Rocktaschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020.

[2] A. Salve, M. Deshmukh, S. Attar, S. Shivpuje, and A. M. Utsab, "A Collaborative Multi-Agent Approach to Retrieval-Augmented Generation across Diverse Data Sources," iASYS Technology Solutions Pvt. Ltd. / SVPM's College of Engineering / BRAC University, 2024.

[3] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in Proceedings of ICLR, 2024.

[4] Z. Wang, Z. Wang, L. Le, H. Zheng, S. Mishra, V. Perot, and T. Pfister, "Speculative RAG: Enhancing Retrieval-Augmented Generation through Drafting," arXiv preprint arXiv:2407.08223, 2024.

[5] X. Zhang, M. Wang, X. Yang, D. Wang, S. Feng, and Y. Zhang, "Hierarchical Retrieval-Augmented Generation Model with Rethink for Multi-Hop Question Answering," arXiv preprint arXiv:2408.11875, 2024.

[6] S. Wang, Y. Fang, Y. Zhou, X. Liu, and Y. Ma, "ArchRAG: Attributed Community-Based Hierarchical Retrieval-Augmented Generation," arXiv preprint arXiv:2502.09891, 2025.

[7] S. Zeng, J. Zhang, P. He, Y. Xing, Y. Liu, H. Xu, J. Ren, S. Wang, D. Yin, and Y. Chang, "The Good and the Bad: Exploring Privacy Issues in Retrieval-Augmented Generation (RAG)," arXiv preprint arXiv:2402.16893, 2024.

[8] C. Xu, Y. Miao, T. H. Luan, P. W. Eklund, Y. Xiang, and L. Gao, "Distributed Retrieval-Augmented Generation," Victoria University, Melbourne, Australia, 2024.

[9] P. Addison, M.-T. H. Nguyen, T. Medan, M. T. Manzari, B. McElrone, L. Lalwani, A. More, S. Sharma, H. R. Roth, and I. Yang, "C-FedRAG: A Confidential Federated Retrieval-Augmented Generation System," arXiv preprint arXiv:2412.13163, 2024.

[10] T. Koga, R. Wu, and K. Chaudhuri, "Privacy-Preserving Retrieval Augmented Generation with Differential Privacy," arXiv preprint arXiv:2412.04697, 2024.

[11] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, "A Comprehensive Overview of Large Language Models," arXiv preprint arXiv:2307.06435, 2024.

[12] Y. Qu, Y. Dai, S. Yu, P. Tanikella, T. Schrank, T. Hackman, D. Li, and D. Wu, "A Novel Compact LLM Framework for Local, High-Privacy EHR Data Applications," arXiv preprint arXiv:2412.02868, 2024.

[13] M. Pourreza and D. Rafiei, "DTS-SQL: Decomposed Text-to-SQL with Small Large Language Models," in Findings of the Association for Computational Linguistics: EMNLP 2024, pp. 8212–8220, November 2024.

[14] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers," in Proceedings of ACL, 2020.

[15] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction," in Advances in Neural Information Processing Systems, vol. 36, 2023.

[16] A. Ziletti and L. D'Ambrosi, "Retrieval Augmented Text-to-SQL Generation for Epidemiological Question Answering using Electronic Health Records," arXiv preprint arXiv:2403.09226, 2024.

[17] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, "Cognitive Architectures for Language Agents," arXiv preprint arXiv:2309.02427, 2024.

[18] Y. Talebirad and A. Nadiri, "Multi-Agent Collaboration: Harnessing the Power of Intelligent LLM Agents," arXiv preprint arXiv:2306.03314, 2023.

[19] Anonymous Authors, "HAWK: A Structured Multi-Agent Workflow Framework for Agent Coordination, Task Scheduling, and Resource Orchestration," 2025.

[20] M. Finsas and J. Maksim, "Optimizing RAG Systems for Technical Support with LLM-Based Relevance Feedback and Multi-Agent Patterns," Master's Thesis, NTNU, 2024.

[21] F. Jiang, Y. Peng, L. Dong, K. Wang, K. Yang, C. Pan, and O. A. Dobre, "Large Language Model Enhanced Multi-Agent Systems for 6G Communications," IEEE Wireless Communications, 2024.

[22] E. Kandogan, S. Rahman, N. Bhutani, D. Zhang, R. L. Chen, K. Mitra, S. Gurajada, P. Pezeshkpour, H. Iso, Y. Feng, et al., "A Blueprint Architecture of Compound AI Systems for Enterprise," arXiv preprint arXiv:2406.00584, 2024.

[23] T. Shnitzer, A. Ou, M. Silva, K. Soule, Y. Sun, J. Solomon, N. Thompson, and M. Yurochkin, "Large Language Model Routing with Benchmark Datasets," in First Conference on Language Modeling, 2024.

[24] F. Mu, L. Zhang, Y. Jiang, W. Li, Z. Zhang, P. Xie, and F. Huang, "Unsupervised Query Routing for Retrieval Augmented Generation," arXiv preprint arXiv:2501.07793, 2025.